

Esercitazione 4 – Contenitori + Generics

Classe `DiscreteAttribute`

Sostituire lo array `String values[]` con il Set `Set<String> values=new TreeSet<String>()`

Implementare l'interfaccia `Iterable<String>`

(fornita dal docente)

Modificare le classi che usano `discreteAttribute` di conseguenza.

Classe `SplitNode`

Sostituire l'array `mapSplit[]` di oggetti `SplitInfo` con un contenitore `ArrayList`

`List<SplitInfo> mapSplit=new ArrayList<SplitInfo> ;`

Modificare tutti i metodi che utilizzano `mapSplit`. Modificare tutti i metodi che utilizzano

Classe `Data`

Sostituire l'array `explanatorySet[]` di oggetti `Attribute` con un contenitore

`List<Attribute> explanatorySet = new LinkedList<Attribute>();`

Modificare conseguentemente tutti i metodi che utilizzano `explanatorySet`

Classe `SplitNode`

Modificare la classe `SplitNode` in modo da implementare l'interfaccia `Comparable<SplitNode>` per realizzare il confronto tra oggetti di tipo `SplitNode` rispetto all'information gain.

Ri-definire il metodo `compareTo`:

`public int compareTo(SplitNode o)`

Input: Nodo di split da confrontare con il corrente nodo `DiscreteNode`

Output: Esito del confronto (**0**: uguali, **-1** gain minore, **1** gain maggiore)

Comportamento: Confrontare i valori di `splitVariance` dei due nodi e restituire l'esito

Classe `RegressionTree`

Modificare il metodo `determineBestSplitNode()` usando il contenitore `TreeSet` per la collezione degli oggetti `SplitNode` generati rispetto un attributo

```
TreeSet <SplitNode> ts = new TreeSet<SplitNode>();
```

Usare la caratteristica dei contenitori **TreeSet** per selezionare il miglior split tra tutti i possibili split ordinati rispetto alla splitVariance.

*NOTA: **Classe TreeSet**: Gestione di insiemi ordinati mediante alberi binari di ricerca non bilanciati. La differenza con la classe HashSet è quindi nella possibilità di mantenere un ordine totale. Gli oggetti dell'insieme devono fornire il metodo **compareTo** dell'interfaccia standard **Comparable** per mantenere le chiavi ordinate.*

Le modifiche apportate non alterano l'output della esecuzione.